



chapter 8

■ ■ ■ ■ ■ ■ ■ ■ ■ ■

结构和联合

1

结构的定义和使用

2

结构数组与结构指针

3

结构在函数间的传递

4

位字段结构

5

联合

6

类型定义语句typedef

7

枚举类型

结构的定义

- 在标准C语言中，使用结构可以把**不同类型**的数据存储在一起
- 在C语言程序中先定义结构，才能进行结构变量的定义和使用
- 结构是由不同数据类型的数据组成的。组成结构的每个数据称为该结构的成员项，简称成员
- 在程序中使用结构，先要对结构的组成进行描述，称为结构的定义

```
struct 结构名
{
    数据类型  成员名1;
    数据类型  成员名2;
    .....
    数据类型  成员名n;
};
```

结构定义规则

- 结构的定义以关键字**struct**作为标识符，其后是定义的结构名，两者为特定结构的类型标识符。结构名命名原则与变量名相同
- 每个成员项由其数据类型和成员名组成。每个成员项后用分号 “;” 作为结束符。整个结构的定义也用分号作为结束符

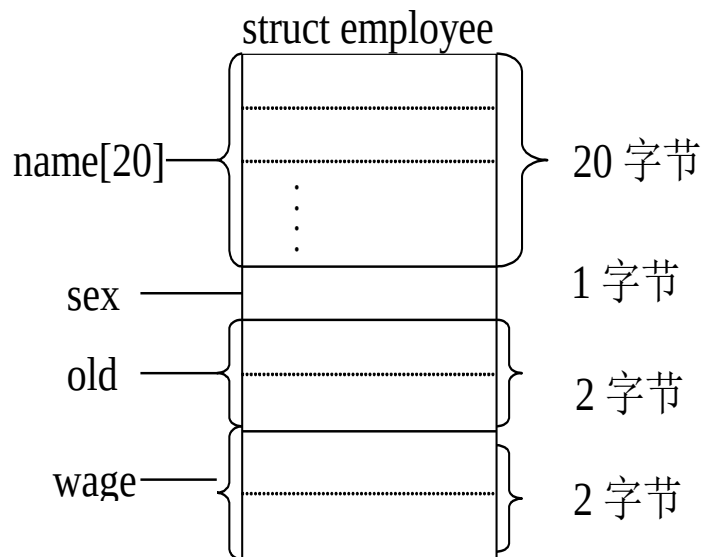
```
struct Employee
{
    char name[20];
    char sex;
    int old;
    int wage;
};
```

结构定义规则

- 结构的定义明确地描述了其组织形式。在程序执行时，结构的定义并不引起系统为该结构分配内存空间
- 结构Employee的定义，仅仅指定了在使用这种结构时应该按下图所示的配置情况占用内存，但这时并没有实际占用内存空间。



```
struct Employee
{
    char name[20];
    char sex;
    int old;
    int wage;
};
```



程序一旦定义了一个结构体，就相当于定义了一个新的结构类型，那么就可以把结构名当作像int、double等关键字一样使用，用说明语句定义该形式结构体的具体结构变量，其格式为：

```
<存储类型> struct 结构名 结构变量名;
```

结构变量的定义在程序的数据说明部分给出：

```
struct Employee wang;
```

结构变量的定义将引起系统按照结构定义时制定的内存模式，为被定义的结构变量分配一定的内存空间。当多个结构变量使用结构时，它们可以在一起定义：

```
struct Employee wang, li,zhang;
```

结构变量使用内存空间，具有一定的存储类型。结构变量的存储类型、寿命、可见性及使用范围与普通变量、数组完全一致

8.1.2

结构变量的定义

结构变量的定义在该结构的定义之后，也可以同时进行，被定义的结构变量直接在结构定义的大括号 } 后给出

```
struct Employee
{
    char name[20];
    char sex;
    int old;
    int wage;
} wang ,song ,zhou;
```



```
struct Employee
{
    char name[20];
    char sex;
    int old;
    int wage;
};
struct Employee wang ,song ,zhou;
```

一个结构变量占用内存的实际大小，可以利用sizeof运算求出。

```
1  #include <stdio.h>
2  void main()
3  { struct Employee
4  {
5      char name[20];
6      char sex;
7      int old;
8      int wage;
9  } wang;
10 printf("%d\n",sizeof(struct Employee));
11 printf("%d\n",sizeof(wang));
12 }
```

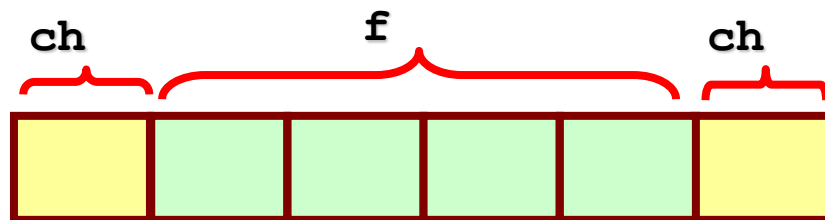
```
32
32
```


8.1.2

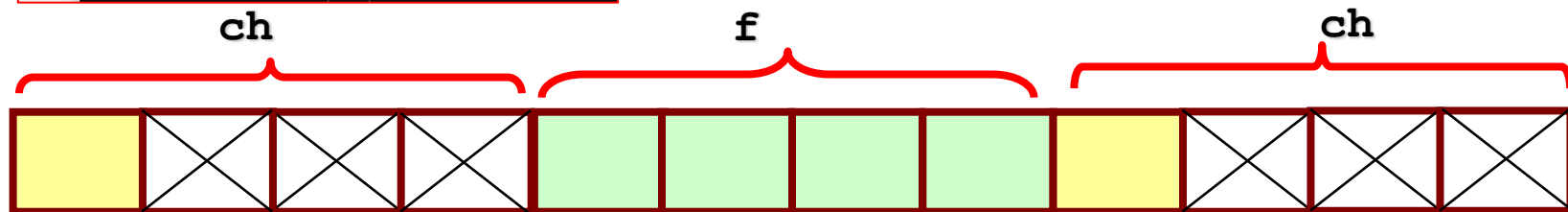
结构变量的定义

```
1  #include <stdio.h>
2  typedef struct test
3  {
4      char a;
5      float b;
6      char c;
7  } SAMPLE;
9  void main()
10 { printf("%d", sizeof(SAMPLE));
11 }
12
```

D:\tmp\c-coding\test\bin\Debug\test.exe



非所有成员变量的内存总和



字节对齐的规则:

- 结构体变量的首地址能够被其最宽基本类型成员的大小所整除;
- 结构体每个成员相对于结构体首地址的**偏移量**都是当前成员大小的整数倍, 如有需要编译器会在成员之间加上填充字节;
- 结构体的总大小为结构体**最宽基本类型**成员大小的整数倍, 如有需要编译器会在最末一个成员之后加上填充字节。

结构是不同数据类型的若干数据的集合体。在程序中使用结构时，一般情况下不能把结构作为一整体参加数据处理，而是使用结构的各个成员项数据来参加各种运算和操作。

结构的成员项一般用以下形式表示：

结构变量名.成员名

结构变量wang具有下列四个成员项：

wang.name

wang.sex

wang.old

wang.wage

```
struct Employee
{
    char name[20];
    char sex;
    int old;
    int wage;
} wang;
```

8.1.3

结构变量的使用形式和初始化

例8.1 结构的使用形式

```
#include <stdio.h>
//结构的定义
struct Employee
{
    char *name;
    char sex;
    int old;
    char *tel;
    char *adr;
};
void main()
{
    //结构变量的定义
    struct Employee wang,gao;
    //结构变量的成员赋值
    wang.name="wang hai";
    wang.sex='M';
    wang.old=34;
    wang.tel="010-12345678";
    wang.adr="beijing";
```

```
    gao.name="gao yang";
    gao.sex='F';
    gao.old=42;
    gao.tel="021-87654321";
    gao.adr="shanghai";
    //显示结构变量的成员内容
    printf(" name sex old tel address\n");
    printf("                                \n");
    printf("%-14s%-4c%-4d%-10s%-20s\n",\
        wang.name,wang.sex,wang.old,\
        wang.tel,wang.adr);
    printf("%-14s%-4c%-4d%-10s%-20s\n",\
        gao.name,gao.sex,gao.old,\
        gao.tel,gao.adr);
}
```

运行结果:

name	sex	old	tel	adr
wang hai	m	34	010-12345678	beijing
gao yang	f	42	021-87654321	shanghai

结构的初始化

在结构说明的同时给这个结构的每个成员赋初值

```
struct 结构名 结构变量={初始数据};
```

初始数据之间用逗号分隔，初始数据的个数与结构成员项的个数应该相同。每个初始数据必须符合与其对应的成员项的数据类型。

```
struct Employee wang={"wang hai",'M',34,"123-1111","beijing"};
```

其功能等价于

```
wang.name="wang hai";  
wang.sex='M';  
wang.old=34;  
wang.tel="010-12345678";  
wang.adr="beijing";
```

结构数组

具有相同结构类型的结构变量也可以组成数组，称它们为结构数组。说明形式：

```
<存储类型> struct 结构名 结构数组名[元素个数] [ = {初值表} ];
```

例如：

```
struct Employee man[3];
```

说明了结构数组man[]，它有三个元素man[0]、man[1]和man[2]，它们都是具有Employee结构的结构变量

- 在定义结构数组的同时可以用初始化列表给它的每个元素赋初值
- 在对结构数组进行初始化时，方括号[]中的元素个数可以缺省
- 结构数组也具有数组的属性，结构数组名是结构数组存储首地址

例8.2 结构数组的使用

```
#include <stdio.h>
#define STUDENT 3
//用符号常量STUDENT表示学生人数
struct Data {           //定义一个结构
    char name[20];      // 姓名
    short age;          // 年龄
    char adr[30];       // 地址
    long tel;           // 电话号码
};
void main( )
{
    struct Data man[STUDENT] = {
        //定义一个结构数组并初始化
        {"王伟", 20, "东八舍416室", 87543641},
        {"李玲", 21, "南三舍219室", 87543945},
        {"张利", 19, "东八舍419室", 87543645}};
    int i;
```

```
// 输出显示表头提示信息
printf("编号\t姓名\t年龄\t地址\t电话\n\n");
//输出结构数组的数据
for(i = 0; i < STUDENT; i++)
    printf("%-d\t%-s\t%-d\t%-s\t%Ld\n", \
            i+1,man[i].name, man[i].age, \
            man[i].adr,man[i].tel);
// 每个输出项都左对齐, 编号从1开始
printf("\n结构类型Data的数据长度:\n
        %d字节。 \n", sizeof(struct Data));
}
```

运行结果:

编号	姓名	年龄	地 址	电 话
1	王伟	20	东八舍416室	87543641
2	李玲	21	南三舍219室	87543945
3	张利	19	东八舍419室	87543645

结构类型Data的数据长度: 56字节

例8.3 对候选人得票进行统计: 设有4个候选人, N个人参加选举, 每次输入一个得票的候选人的名字, 要求最后输出4个候选人的得票结果。

```
#include <stdio.h>
#include <string.h>
//宏定义, 定义参加选举人的个数
#define N 10
//结构定义
struct person
{
    char name[20];
    int count;
};
void main()
{
    //结构数组定义及初始化
    struct person leader[4]={\
        {"wang",0},{ "zhang",0},\
        {"zhou",0},{ "gao",0}};
    char name[20],i,j;
```

```
//模拟选举过程, 循环一次代表一次选举
for(i=0;i<N;i++)
{
    gets(name);
    //在候选人中查找匹配的人
    for(j=0;j<4;j++)
        if(strcmp(name,leader[j].name)==0)
        {
            leader[j].count++;
            break;
        }
    printf("\n");
    for(j=0;j<4;j++)
        printf("%s:%d\n",leader[j].name,\
            leader[j].count);
}
```

运行结果:
//输入
wang
wang
zhang
zhang
zhang
zhang
zhou
zhou
gao
gao
//输出
wang:2
zhang:4
zhou:2
gao:2

指向结构的指针变量称为结构指针。结构指针的运算按地址计算规则进行，定义格式为(结构名必须是已经定义过的结构类型)：

```
<存储类型> struct 结构名 * 结构指针名 [ = 初始地址值 ];
```

例如：

```
struct Employee * pman;  
struct Employee * pd;
```

- 存储类型是结构指针变量本身的存储类型
- 由于结构指针是指向整个结构体而不是某个成员,因此结构指针的增(减)量运算，将跳过**一个结构变量的整体**，指向内存中下一个结构变量或结构数组中下一个元素，结构指针本身的（物理）地址增量值取决于它所指的结构变量的数据长度

- 结构体可以嵌套，即结构体成员又是一个结构变量或结构指针

例如

```
struct student {  
    char   name[20];  
    short  age;  
    char   adr[30];  
    struct Date BirthDay;  
    struct Date StudyDate;  
} studt[300];
```

- 在结构嵌套中，当结构体的成员项具有由该结构类型定义的结构指针时，这种结构体称为**递归结构体**

```
struct Node {  
    // 数据场，节点序号  
    int  num;  
    // 指针场，指向下一个节点的结构指针  
    struct Node * next;  
};
```

使用结构指针对于结构成员进行引用时，有两种形式：

- 使用结构成员访问运算符 “.”

(*结构指针名).成员名

运算符“.”的优先级比取内容运算符“*”高，所以需要使用圆括号

- 结构指针运算符 “->”（横线加大于号）。它与前一种表示方法在意义上是完全等价的。

结构指针名->成员名

例8.4 结构指针的使用

```
#include <stdio.h>
struct Date
//定义一个结构
{
    int month;
    int day;
    int year;
};
```

运行结果：
Today is 4/15/1990

```
void main()
{
    //定义一个结构变量和结构指针变量
    struct Date today,*date_p;
    //将结构指针指向一个结构变量
    date_p=&today;
    //采用结构指针给目标变量赋值
    date_p->month=4;
    date_p->day=15;
    date_p->year=1990;
    //采用结构指针输出目标变量的数据
    printf("Today is %d/%d/%d\n", \
        date_p->month,date_p->day, \
        date_p->year );
}
```

例8.5 结构指针运算

```
#include <stdio.h>
struct Key
{
    char *keyword;
    int keyno;
};
void main()
{
    struct Key kd[]={{"are",123},\
                     {"my",456},{"you",789}};
    struct Key *p;
    int a;
    char chr;
    p=kd;
    a=p->keyno;
    printf("p=%d,a=%d\n",p,a);
    a=++p->keyno;
    printf("p=%d,a=%d\n",p,a);
    a=(++p)->keyno;
    printf("p=%d,a=%d\n",p,a);
    a=(p++)->keyno;
    printf("p=%d,a=%d\n",p,a);
```

定义结构

结构指针变量

访问成员

a=++(p->keyno)

变化后指向目标的成员

变化前指向目标的成员

```
p=kd;
chr=*p->keyword;
printf("p=%d,chr=%c(adr=%d)\n",\
p,chr,p->keyword);
chr=*p->keyword++;
printf("p=%d,chr=%c(adr=%d)\n",\
p,chr,p->keyword);
chr=*p++->keyword;
printf("p=%d,chr=%c(adr=%d)\n",\
p,chr,p->keyword);
chr=*++p->keyword;
printf("p=%d,chr=%c(adr=%d)\n",\
p,chr,p->keyword);
}
```

chr=(p->keyword)

chr=((p->keyword)++)

变化前指向目标的成员

chr=(++(p->keyword))

```
p=6421984, a=123
p=6421984, a=124
p=6422000, a=456
p=6422016, a=456
p=6421984, chr=a (adr=4210688)
p=6421984, chr=a (adr=4210689)
p=6422000, chr=r (adr=4210692)
p=6422000, chr=y (adr=4210693)
```

例8.6 指向结构数组的指针

```
#include <stdio.h>
struct student //定义一个结构
{
    int No;
    char name[20];
    char sex;
    int age;
};
void main()
{
    //定义一个结构数组并初始化
    struct student stu[3]={
        {10101,"Li Lin",'M',18},
        {10102,"Zhang fan",'M',19},
        {10104,"Wang  Min",'F',20}};
```

```
//定义一个结构指针变量
struct student *p;
printf("No. Name Sex Age\n");
for( p=stu ; p<stu+3 ; p++)
    printf("%8d%-12s%6c%4d\n",\
        p->No,p->name,p->sex,p->age);
}
```

运行结果:

No.	Name	Sex	Age
10101	Li Lin	M	18
10102	Zhang fan	M	19
10104	Wang Min	F	20

将一个结构体变量的值传递给另一个函数，有3种方法：

- 用结构体变量的**成员作实参**。用法和用普通变量作实参是一样的，属于“**值传递**”方式
- 用**结构体变量作实参**。在调用函数时，把结构作为参数传递给函数，采取的是“值传递”的方式，将结构体变量所占的内存单元的**内容**全部顺序传递给形参——内容复制
- 用结构变量的地址或结构数组的**首地址作为实参**。用指向相同结构类型的结构指针作为函数的形参来接受该地址值

例8.7 采用值传递在函数间传递结构变量，计算雇佣天数和工资。

```

#include <stdio.h>
struct Date {
    int day;
    int month;
    int year;
    int yearday;
    char mon_name[4];
};
int day_of_year(struct Date pd);
void main()
{
    struct Date HireDate;
    float laborage; // 存放每天的雇佣工资
    printf("请输入每天的工资:");
    scanf("%f", &laborage);
    printf("请输入年份:");
    scanf("%d", &HireDate.year);
    printf("请输入月份:");
    scanf("%d", &HireDate.month);
    printf("请输入日:");
    scanf("%d", &HireDate.day);
    HireDate.yearday = day_of_year(HireDate);
    printf("从%d年元月1日到%d年%d月%d\
    日的雇佣期限:%d天, \n应付给你的\
    工钱: %-.2f元. \n", \
    HireDate.year, HireDate.year, HireDate.month, \
    HireDate.day, HireDate.yearday, \
    laborage * HireDate.yearday);
}

```

```

// 计算某年某月某日是这一年的第几天
int day_of_year(struct Date pd)
{
    int day_tab[2][13] = { {0,31,28,31,30,31,30,31,31,30,31,30,31},
                           // 非闰年的每月天数
                           {0,31,29,31,30,31,30,31,31,30,31,30,31}}; // 闰年的每月天数
    /* 为了与月份一致，列号不使用为零的下标号。
    行号为0是非闰年的每月天数，为1是闰年的每月天数 */
    int i, day, leap; // leap为0是非闰年，为1是闰年
    day = pd.day; // 将当月的天数加入
    leap = pd.year % 4 == 0 && // 能被4整除的年份基本上
                           // 是闰年
    pd.year % 100 != 0 || // 能被100整除的年份不是闰年
    pd.year % 400 == 0; // 能被400整除的年份又是闰年
    // 求从元月1日算起到这一天的累加天数
    for(i = 1; i < pd.month; i++)
        day += day_tab[leap][i];
    return day;
}

```

运行结果:

```

请输入每天的工资: 38.5(CR)
请输入年份: 2000(CR)
请输入月份: 10(CR)
请输入日: 1(CR)
从2000年元月1日到2000年10月1日
的雇佣期限: 275天
应付给你的工钱: 10587.50元。

```

例8.8 通信录的建立和显示程序(采用地址方式传递结构变量)

```
#include <stdio.h>
#define MAX 3
struct Record//定义一个通信录的结构
{
    char name[20];
    char addr[50];
    char tel[15];
};
int input(struct Record*pt); //通信录录入函数
void display(struct Record*pt,int n);
                        //通信录显示函数

void main()
{
    struct Record man[MAX]; //定义结构数组
    int i;
    for(i=0;i<MAX;i++)      //建立通信录
        if(input(&man[i])==0)break;
    display(man,i);          //显示通信录
}
```

```
int input(struct Record *pt)
{
    printf("Name?");
    gets(pt->name);
    if(pt->name[0]=='0')return 0;
    printf("Address?");
    gets(pt->addr);
    printf("Telephone?");
    gets(pt->tel);
    return 1;                //正常返回1
}

void display(struct Record *pt,int n)
{
    int i;
    printf(" name          address          tel\n");
    printf("-----\n");
    for(i=0 ; i<n ; i++,pt++)
        printf("%-15s%-30s%s\n",pt->name, \
            pt->addr,pt->tel);
}
```

input()函数用于以人机对话方式输入数据，它的形式参数pt是结构指针，用来接收结构地址

display()函数显示输入结果。调用时，实参是结构数组man[]的首地址

例8.8 通信录的建立和显示程序(采用地址方式传递结构变量)

//输入

该程序运行结果为:

Name?王伟

Address?东八舍416室

Telephone? 87543641

Name?李玲

Address?南三舍219室

Telephone? 87543945

Name?张利

Address?东八舍419室

Telephone?87543645

//输出

name

address

tel

王伟 东八舍416室 87543641

李玲 南三舍219室 87543945

张利 东八舍419室 87543645

例8.9 返回结构变量的函数

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

struct Record          //定义一个结构
{
    char str[20];
    int num;
};

//函数原型声明
struct Record str_add_int(struct Record x);
void main()
{
    struct Record p,s={"31.45",20};
```

运行结果:

31.45 20

51.45 20

```
p=str_add_int(s);
printf("%s  %d\n",s.str,s.num);
printf("%s  %d\n",p.str,p.num);
}

//形参x和实参s的类型一致
struct Record str_add_int(struct Record x)
{
    float e;
    e=atof(x.str);           //将字符串x.str转换为浮
                             //点数并赋给e
    e=e+x.num;               //浮点数与整数相加
    gcvt(e,5,x.str);         //将浮点数e再转换为字
                             //符串，并赋给x.str
    return x;                //将处理后的结果（结构
                             //变量）返回给调用函数
}
```

值传递，将结构变量s传递给函数，处理后再将结果通过return语句返回

例8.10 返回结构指针的函数

```
#include <stdio.h>
#define NULL 0
struct Sample    //定义一个结构
{
    int num;
    char color;
    char type;
};
struct Sample *find(struct Sample *pd,int n);
void main()
{
    int num;
    struct Sample *result; //结构指针变量定义
    struct Sample car[]={101,'G','c'}, {210,'Y','m'},\
        {105,'R','l'}, {222,'B','s'}, {308,'P','b'}, {0,0,0};
    printf("Enter the number:");
    scanf("%d",&num); //输入查找样品代号
    result=find(car,num);
    if(result!=NULL)
    {
        printf("number :%d\n",result->num);
        printf("color :%c\n",result->color);
        printf("type :%c\n",result->type);
    }
}
```

```
else
    printf("not found");//未找到，给出提示信息
}
struct Sample *find(struct Sample *pd,int n)
{
    int i;
    for(i=0 ; pd[i].num!=0 ;i++)
        if(pd[i].num==n) break;    //找到，退出循环
    if(pd[i].num!=NULL)
        return pd+i;                //返回查找结果
    else
        return NULL;
}
```

运行结果：

```
Enter the number:101    //输入
number:101              //输出
color:G
type:c
```

结构指针型函数是以地址传递方式向调用它的函数返回结构的数据。不仅可以返回某个结构的地址，也可以返回结构数组的地址，从而把函数中处理的若干结构的数据返回给调用它的函数中